C++
(Class Concepts)

for

IDC 122

Syllabus

Introduction to Class in C++: Basic notions of procedural and object-oriented programming, the concept of a Class, how to define and implement a Class, Class data members, Class methods, Private & Public member, Creating and using Class objects.

Basic Transition in I/O (C -> C++)

```
int a;
Input: scanf("%d", &a);
Output: printf("%d",a);
```

C

```
int a;
Input: cin>>a;
Output: cout<<a;
```

C++

For string input (must take care):

```
char str[20];
Input: scanf(" %s",str);
Output: printf("%s",str);
```

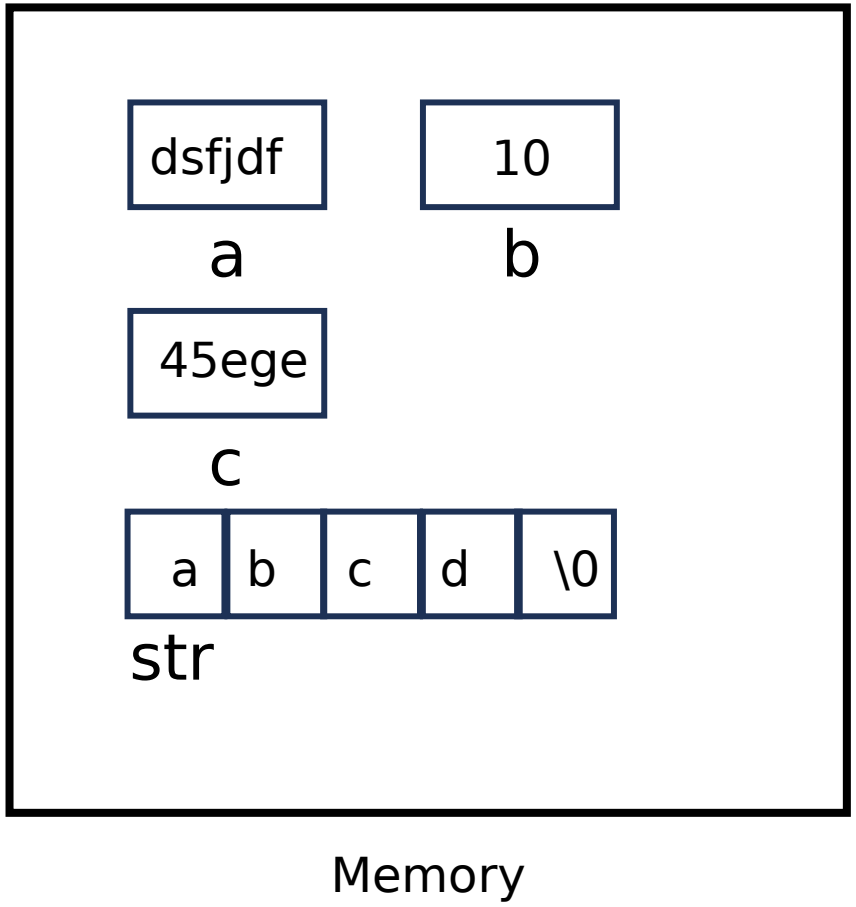
C

```
char str[20];
Input: cin>>ws;
       cin.get(str,20);
Output: cout<<str;
```

C++

Basic Recall on Memory Allocation

```
int main()
{
    int a;
    int b =10;
    float c;
    char
str[5]={‘a’,’b’,’c’,’d’,’\
0’};
}
```



C++ Versions and Their Features

Version	Year	Major Features
C++98	1998	First standard version, introduced STL (Standard Template Library)
C++03	2003	Bug fixes and improvements to C++98, minor refinements
C++11	2011	Major update: Smart pointers, `auto` keyword, lambda expressions, move semantics, `nullptr`, `constexpr`, `std::thread`, range-based for loops, and more
C++14	2014	Improvements to C++11: Generic lambdas, `std::make_unique`, relaxed constexpr, binary literals (`0b...`), and return type deduction
C++17	2017	More powerful constexpr, structured bindings, `std::optional`, `std::variant`, `std::any`, if constexpr, filesystem library, parallel algorithms
C++20	2020	Concepts, modules, coroutines, ranges, `std::span`, three-way comparison (`<=>` spaceship operator), improved constexpr, `std::format`, calendar/time library
C++23	2023	`std::expected`, `std::mdspan`, monadic operations for `std::optional`, better `constexpr`, improvements to modules, new standard library utilities

C++ Versions

1. There are various versions of the C++.
2. Not all features are supported by all the compilers.
3. As of today, up to **C++ 14** features are supported by most of the compilers.

Programming Approaches

1. Procedural/Structured

C, Pascal, Fortran, COBOL, BASIC

2. Object Oriented Programming

C++, PYTHON, JAVA, C#

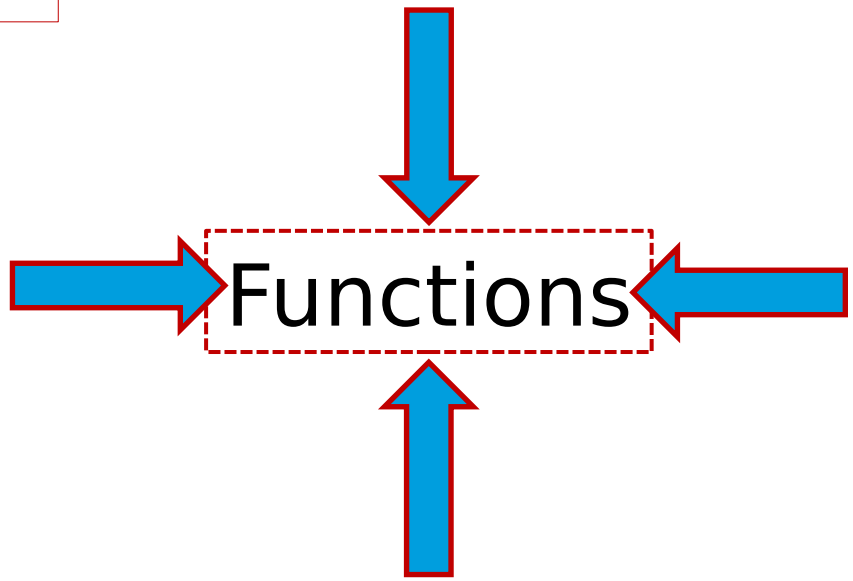
Procedural Vs Object Oriented Programming(1/4)

What is a Procedural programming?

Focus

C

Approach



1. The way we program is more **focused on using functions**.
2. **Data** are **evaluated, passed and returned using functions**.

Procedural Vs Object Oriented Programming(2/4)

```c
#include <stdio.h>

void function1() {
    printf("Inside function1\n");
}

void function2() {
    printf("Inside function2\n");
}
int main() {
    printf("Inside main function\n");
    function1();
    function2();
    printf("Back in main function\n");
    return 0;
}
```

We **approach/*think*** in terms of functions.

Disadvantage: Maintaining, debugging and upgrading **large programs** were a difficult task.

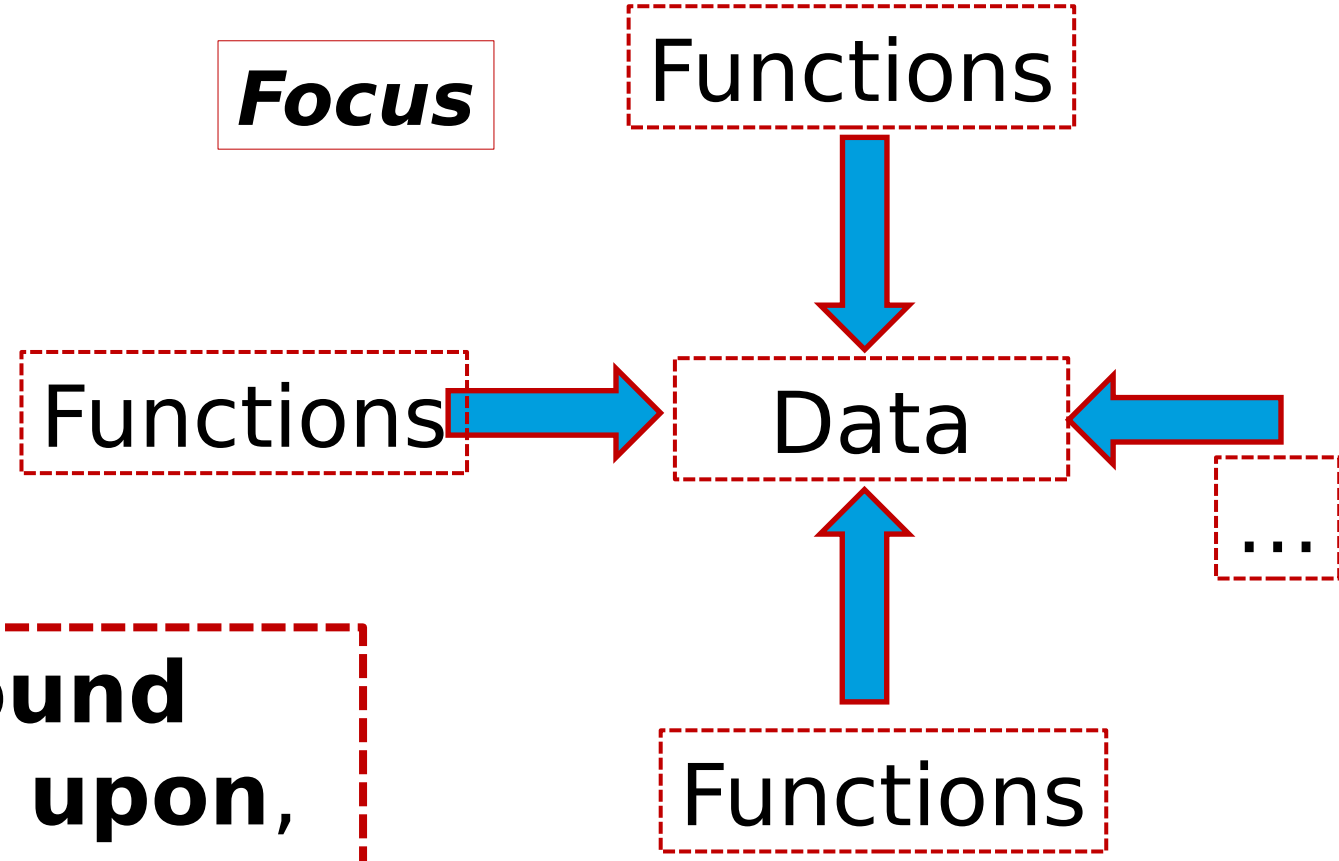
Procedural Vs Object Oriented Programming(3/4)

What is a Object Oriented programming?

C++

Approach

Focus

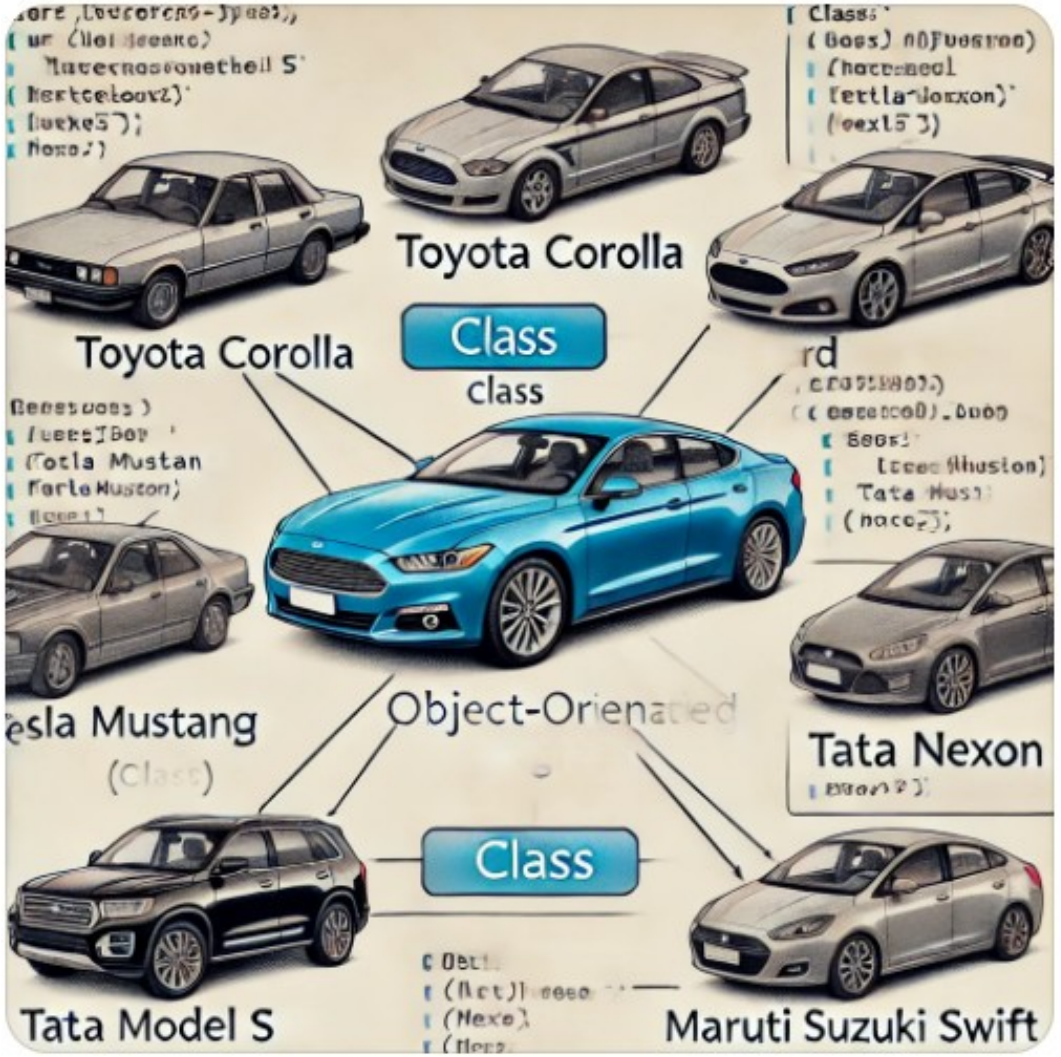


Program is designed around the data being operated upon, rather than around the operations themselves.

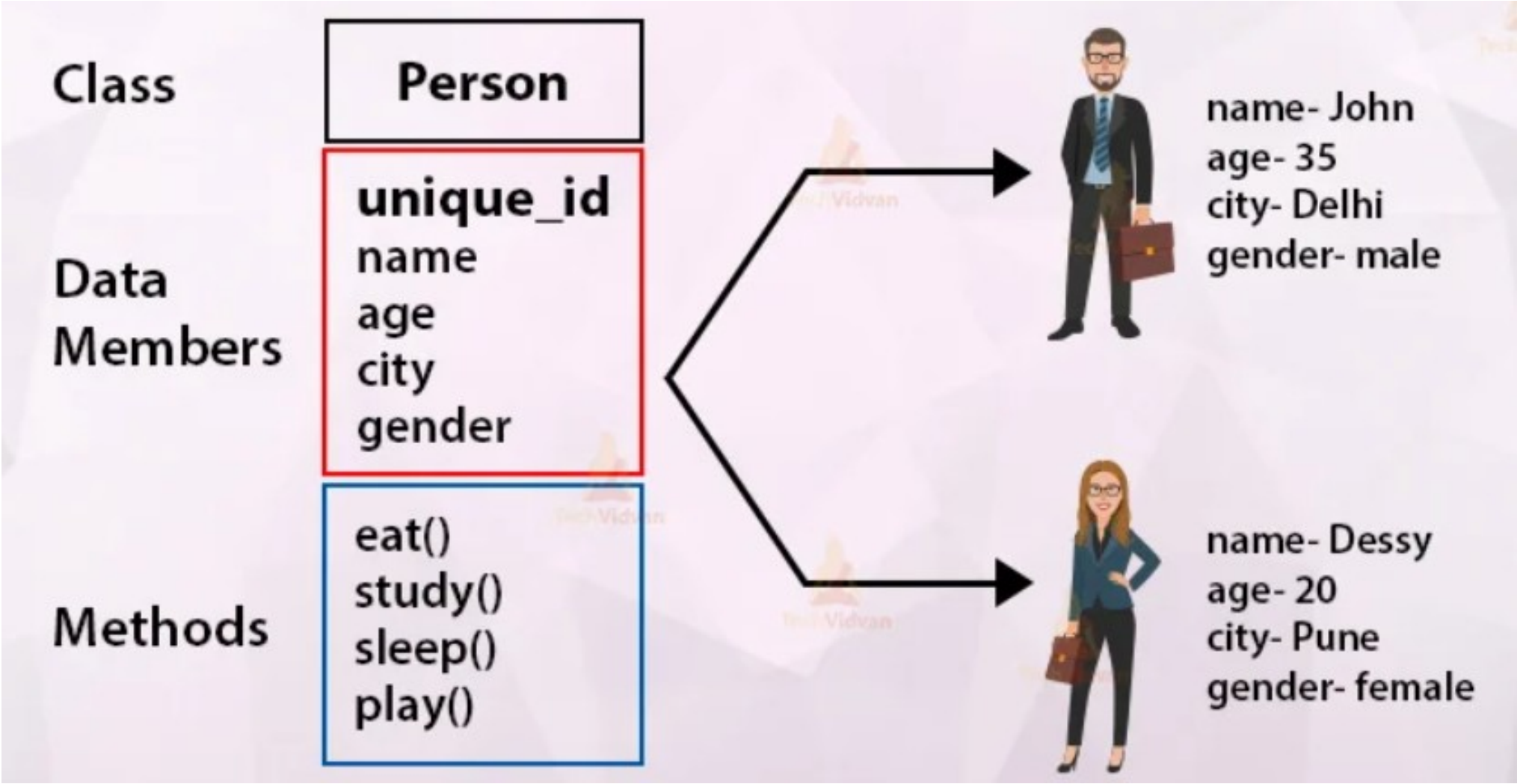
Procedural Vs Object Oriented Programming(4/4)

Object Oriented Programming (OOP)

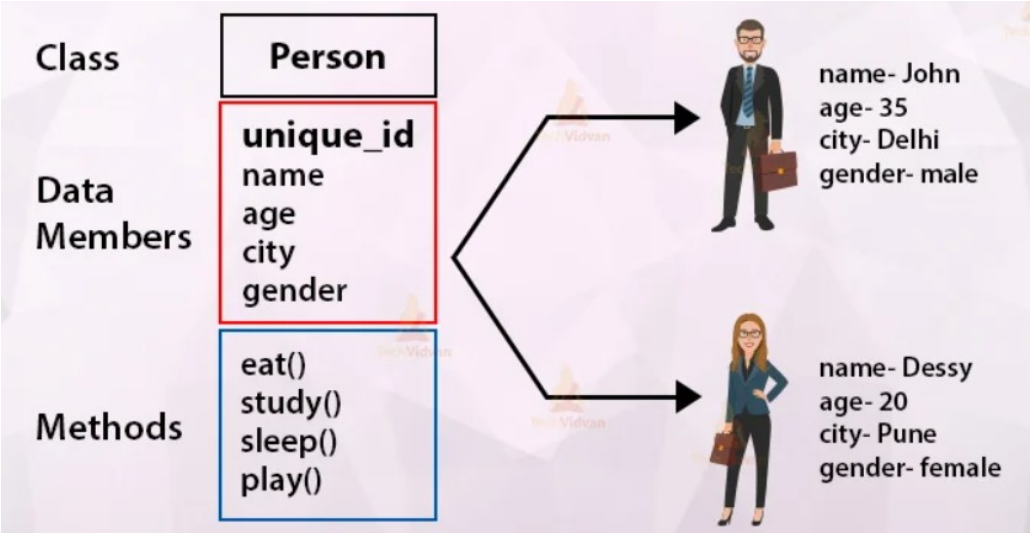
1. To achieve the focus on data OOPs uses the concept of **Class** & **Objects**.
2. Moreover, **Objects model the real world effectively into the programs**.
3. Program design become easy.
4. Maintaining, debugging and upgrading **large programs** become possible.



Object Oriented Programming



Class C++ Declarat



Classes are **prototype** and its **instances** are Objects.

Access Specifiers: **private** and **public.**
Private members can be accessed only by the class members.

```
class Person {
private:
    int unique_id;
    string name;
    int age;
    string city;
    string gender;

public:
    // Constructor
    Person(int id, string n, int a, string c, string g);

    // Method declarations
    void eat();
    void study();
    void sleep();
    void play();
    void displayInfo();
};
```

Data Members

Member Functions

C++ Class Declaration

Objects

```cpp
#include <iostream>
#include <string>
using namespace std;

class Person {
private:
    int unique_id;
    string name;
    int age;
    string city;
    string gender;

public:
    // Default constructor (No parameters)
    Person() {
        unique_id = 0;
        name = "Unknown";
        age = 0;
        city = "Not Assigned";
        gender = "Not Specified";
    }

    // Method declarations
    void displayInfo();
};
```

No **.h** in C++.

namespace is new concept

Access Specifier private members are only accessible by its members.

Data members

Access Specifier public members are accessible outside the class.

Constructors. Initialize the object and have same name class name.

Function declaration

Function definition

```cpp
void Person::displayInfo() {
    cout << "ID: " << unique_id << "\nName: " << name << "\nAge: " << age
         << "\nCity: " << city << "\nGender: " << gender << endl;
    cout << "-------------------------------------" << endl;
}

int main() {
    // Creating two objects using the default constructor
    Person person1;
    Person person2;

    // Displaying their default information
    cout << "Person 1 Info:" << endl;
    person1.displayInfo();

    cout << "Person 2 Info:" << endl;
    person2.displayInfo();

    return 0;
}
```

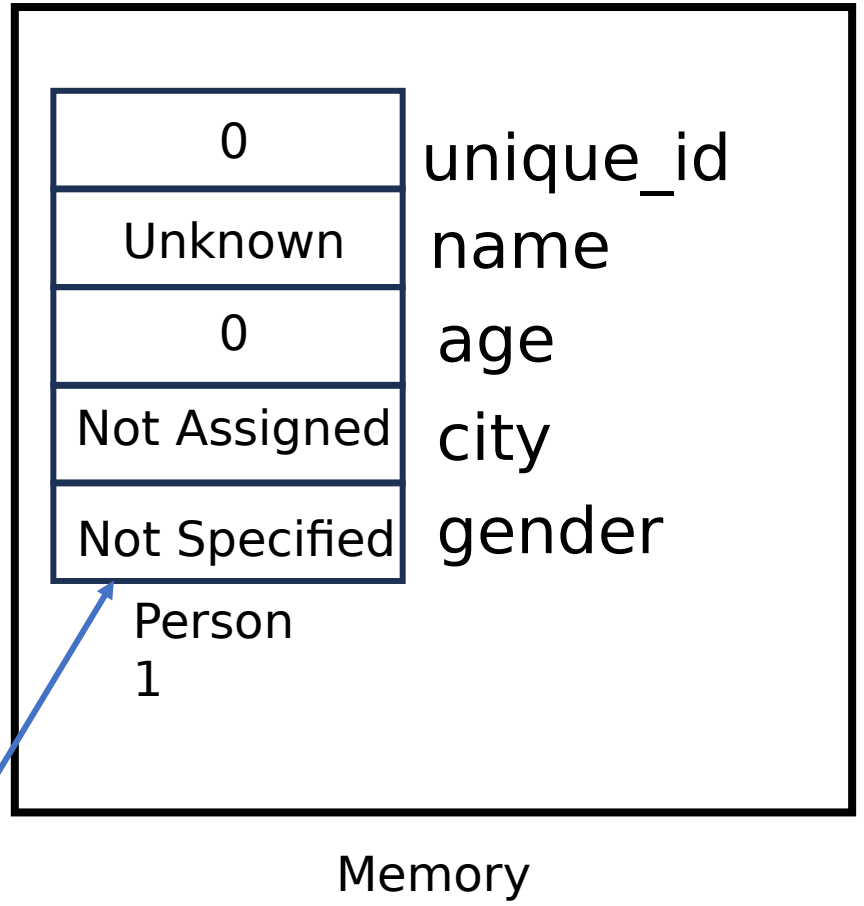
Object creation; internally it is:

Person person1 = Person();

Function call and display data

Constructor (Default)

1. Used to initialize the Object.
Have same name as the class name.
Always invoked during object creation.
2. Default constructor have no parameter.




```
// Default constructor (No parameters)
Person() {
    unique_id = 0;
    name = "Unknown";
    age = 0;
    city = "Not Assigned";
    gender = "Not Specified";
}
```

```
// Creating two objects using the default constructor
Person person1;
Person person2;
```

Constructor (Parametrized)

1. Used to initialize the Object.
2. Have same name as the class name.

```
class Person {
private:
    int unique_id;  // Private data member, assuming it as a unique identifier
public:
    string name;
    int age;
    string city;
    string gender;

    // Constructor
    Person(int id, string n, int a, string c, string g) {
        unique_id = id;
        name = n;
        age = a;
        city = c;
        gender = g;
    }
```

Parameterized Constructor

Object Creation using Parameterized Constructor

```cpp
#include <iostream>
using namespace std;
class Car {  // Class is a prototype
public:
    string brand;
    int year;

    // Constructor
    Car(string b, int y) {
        brand = b;
        year = y;
    }

    void display() {
        cout << "Brand: " << brand;
        cout << ", Year: " << year << endl;
    }
};
```

```cpp
int main() {
    // Creating multiple objects from the same class
    Car car1("Toyota", 2022);
    Car car2("Ford", 2019);
    Car car3("Honda", 2021);

    // Calling method for each object
    car1.display();
    car2.display();
    car3.display();

    return 0;
}
```

Properties of OOPs

Property	Description
Encapsulation	Bundling of data and methods that operate on the data into a single unit (class). It restricts direct access to some of the object's components, ensuring data security.
Abstraction	Hiding complex implementation details and showing only essential features of an object to the user. Helps in reducing complexity.
Inheritance	Allows a new class (child) to acquire the properties and behavior of an existing class (parent). Supports code reusability and hierarchical relationships.
Polymorphism	Enables one interface to be used for different data types. It allows methods to have multiple implementations, such as function overloading and method overriding.
Data Hiding	Restricts access to certain details of an object, preventing unintended interference and ensuring controlled access to data. Achieved using access modifiers like `private`, `protected`, and `public` in C++.